

BAB 1

PENDAHULUAN

1.1 Latar Belakang

1.2 Rumusan Masalah

Rumusan masalah dari tugas akhir ini adalah sebagai berikut:

1. Bagaimana menganalisis, merancang, dan membangun model reinforcement learning untuk permainan Connect Four menggunakan algoritma Q-Deep Network?
2. Bagaimana melatih dan mengukur performa model reinforcement learning untuk permainan Connect Four?
3. Bagaimana memilih model terbaik yang akan digunakan sebagai agen A.I. yang dapat melawan manusia pada permainan Connect Four?

1.3 Tujuan

Tujuan dari tugas akhir ini adalah sebagai berikut:

1. Menganalisis, merancang, dan membangun model reinforcement learning untuk permainan Connect Four menggunakan algoritma Q-Deep Network.
2. Melatih dan mengukur performa model reinforcement learning untuk permainan Connect Four.
3. Memilih model terbaik yang akan digunakan sebagai agen A.I. yang dapat melawan manusia pada permainan Connect Four.

1.4 Batasan Masalah

Untuk mempermudah pembuatan template ini, tentu ada hal-hal yang harus dibatasi, misalnya saja bahwa template ini bukan berupa style L^AT_EX pada umumnya (dengan alasannya karena belum mampu jika diminta membuat seperti itu)

1.5 Metodologi

Tentunya akan diisi dengan metodologi yang serius sehingga templatennya terkesan lebih serius.

Morbi luctus, wisi viverra faucibus pretium, nibh est placerat odio, nec commodo wisi enim eget quam. Quisque libero justo, consectetuer a, feugiat vitae, porttitor eu, libero. Suspendisse

1 sed mauris vitae elit sollicitudin malesuada. Maecenas ultricies eros sit amet ante. Ut venenatis
2 velit. Maecenas sed mi eget dui varius euismod. Phasellus aliquet volutpat odio. Vestibulum ante
3 ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Pellentesque sit amet pede
4 ac sem eleifend consectetur. Nullam elementum, urna vel imperdiet sodales, elit ipsum pharetra
5 ligula, ac pretium ante justo a nulla. Curabitur tristique arcu eu metus. Vestibulum lectus. Proin
6 mauris. Proin eu nunc eu urna hendrerit faucibus. Aliquam auctor, pede consequat laoreet varius,
7 eros tellus scelerisque quam, pellentesque hendrerit ipsum dolor sed augue. Nulla nec lacus.

8 1.6 Sistematika Pembahasan

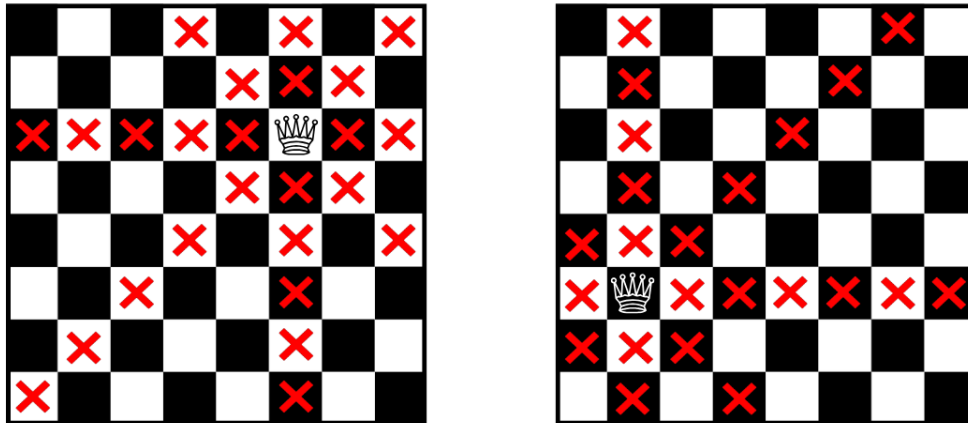
9 Rencananya Bab 2 akan berisi petunjuk penggunaan template dan dasar-dasar L^AT_EX. Mungkin
10 bab 3,4,5 dapt diisi oleh ketiga jurusan, misalnya peraturan dasar skripsi atau pedoman penulisan,
11 tentu jika berkenan. Bab 6 akan diisi dengan kesimpulan, bahwa membuat template ini ternyata
12 sungguh menghabiskan banyak waktu.

BAB 2

LANDASAN TEORI

2.1 Permasalahan N-Queens

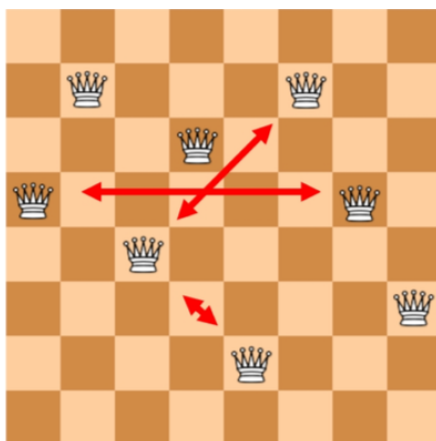
Permasalahan N-Queens merupakan salah satu permasalahan klasik dalam bidang ilmu komputer dan matematika diskrit, dengan sejarah panjang sejak pertama kali diperkenalkan oleh Max Bezzel pada tahun 1848. Pada formulasi dasar permasalahan ini, diberikan sebuah papan catur berukuran $n \times n$ dan tugasnya adalah menempatkan n buah bidak menteri sedemikian rupa sehingga tidak ada dua menteri yang saling menyerang. Sebagaimana dalam aturan permainan catur, sebuah menteri dapat bergerak secara bebas dalam arah horizontal, vertikal, ataupun diagonal dengan jarak tak terbatas, sehingga setiap penempatan menteri harus mempertimbangkan seluruh arah serangan tersebut seperti yang tertera pada Gambar 2.1.



Gambar 2.1: Contoh aturan penyerangan bidak menteri pada permainan catur. Silang merah menandakan kotak yang terserang oleh bidak menteri.

Dari perspektif komputasi, permasalahan ini menarik karena memiliki struktur yang sederhana namun ruang solusinya sangat besar dan tumbuh secara eksponensial. Jika setiap baris diasosiasikan dengan satu menteri, maka setiap menteri memiliki n kemungkinan kolom, menghasilkan ruang solusi awal sebesar n^n . Ketika nilai n meningkat, ruang solusi menjadi sangat luas sehingga pencarian solusi tanpa teknik penyempitan ruang menjadi tidak realistis. Sebagai contoh, untuk $n = 20$ saja terdapat 20^{20} kemungkinan konfigurasi awal, jumlah yang jauh melampaui kemampuan komputasi

1 brute-force konvensional.



Gambar 2.2: Contoh solusi salah masalah N-Queens pada papan berukuran 8×8 (8-Queens)

Sumber: [https://www.researchgate.net/figure/](https://www.researchgate.net/figure/N-Queen-problem-explanation-with-8-queens-in-a-chessboard-of-8_fig3_372415823)

[N-Queen-problem-explanation-with-8-queens-in-a-chessboard-of-8_fig3_372415823](https://www.researchgate.net/figure/N-Queen-problem-explanation-with-8-queens-in-a-chessboard-of-8_fig3_372415823)

2 Secara struktural, permasalahan N-Queens memiliki tiga komponen kendala utama yang harus
3 dipenuhi secara simultan. Pertama, kendala kolom yang memastikan tidak ada dua menteri pada
4 kolom yang sama. Kedua, kendala diagonal positif yang mencegah konflik pada diagonal dengan
5 gradien positif. Ketiga, kendala diagonal negatif yang mencegah konflik pada diagonal dengan
6 gradien negatif. Ketiga kendala ini membentuk sistem pembatasan yang saling berinteraksi, dan
7 pelanggaran terhadap salah satu kendala mengakibatkan konfigurasi menjadi tidak valid. Pada
8 contoh Gambar 2.2, ketiga kendala dilanggar, sehingga menjadi salah satu solusi yang tidak valid
9 untuk permasalahan N-Queens pada papan berukuran 8×8 (8-Queens). Interaksi kompleks
10 antarkendala inilah yang membuat permasalahan ini dapat dimodelkan menggunakan kerangka
11 kerja formal tertentu, yang akan dibahas lebih lanjut pada bagian berikutnya.

12 Studi literatur juga menekankan bahwa meskipun solusi untuk N-Queens selalu ada untuk
13 $n \geq 4$, struktur solusinya sangat beragam dan sering kali memiliki pola-pola tertentu yang dapat
14 dimanfaatkan. Misalnya, penelitian yang dilakukan oleh Hoffman, Loessi, dan Moore¹ yang dikutip
15 dalam survei komprehensif Bell dan Stevens² menjelaskan bahwa terdapat konstruksi aritmatika
16 eksplisit yang dapat menghasilkan solusi dalam waktu linier $O(n)$ tanpa perlu melakukan eksplorasi
17 ruang pencarian (*search space*) sama sekali. Namun, pendekatan konstruktif deterministik semacam
18 ini tidak berlaku secara umum untuk seluruh varian N-Queens—terutama yang memiliki kendala
19 ireguler seperti pada *Colored Queens*—sehingga algoritma pencarian tetap menjadi pendekatan
20 yang paling banyak digunakan. Selain itu, jumlah solusi unik untuk N-Queens tumbuh secara
21 eksponensial seiring bertambahnya n , dengan pola pertumbuhan yang telah menjadi objek kajian
22 dalam teori kombinatorik.

23 Secara keseluruhan, pemahaman mendalam mengenai sifat dasar N-Queens, karakteristik ruang
24 pencariannya, struktur kendala yang mendasarinya, serta berbagai teknik algoritmik yang digunakan
25 untuk menyelesaikannya memberikan landasan konseptual kuat sebelum mempelajari varian-varian

¹E. J. Hoffman, J. C. Loessi, and R. C. Moore, “Constructions for the Solution of the m Queens Problem,” *Mathematics Magazine*, vol. 42, no. 2, hal. 66–72, 1969.

²J. Bell and B. Stevens, “A survey of known results and research areas for n -queens,” *Discrete Mathematics*, vol. 309, no. 1, hal. 1–31, 2009.

- 1 yang lebih kompleks, termasuk Colored Queens yang menjadi fokus tugas akhir ini.

DAFTAR REFERENSI

LAMPIRAN A

KODE PROGRAM

Kode A.1: MyCode.c

```
1 // This does not make algorithmic sense,
2 // but it shows off significant programming characters.
3
4 #include<stdio.h>
5
6 void myFunction( int input, float* output ) {
7     switch ( array[i] ) {
8         case 1: // This is silly code
9             if ( a >= 0 || b <= 3 && c != x )
10                 *output += 0.005 + 20050;
11             char = 'g';
12             b = 2^n + ~right_size - leftSize * MAX_SIZE;
13             c = (--aaa + &daa) / (bbb++ - ccc % 2 );
14             strcpy(a,"hello_$@?");
15         }
16         count = ~mask | 0x00FF00AA;
17     }
18 }
19
20 // Fonts for Displaying Program Code in LATEX
21 // Adrian P. Robson, nepsweb.co.uk
22 // 8 October 2012
23 // http://nepsweb.co.uk/docs/progfonts.pdf
```

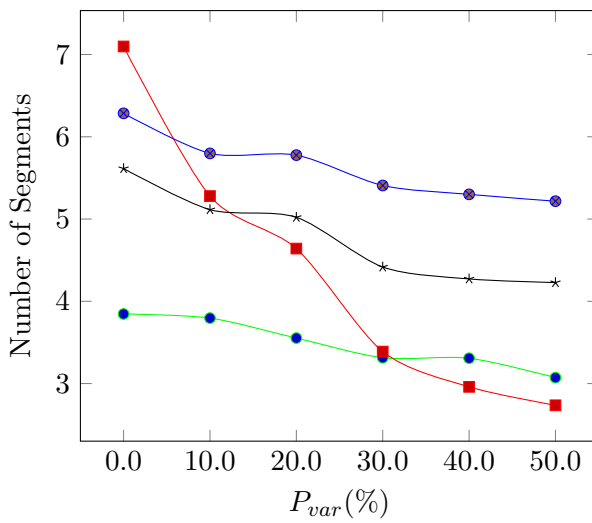
Kode A.2: MyCode.java

```
1 import java.util.ArrayList;
2 import java.util.Collections;
3 import java.util.HashSet;
4
5 //class for set of vertices close to furthest edge
6 public class MyFurSet {
7     protected int id; //id of the set
8     protected MyEdge FurthestEdge; //the furthest edge
9     protected HashSet<MyVertex> set; //set of vertices close to furthest edge
10    protected ArrayList<ArrayList<Integer>> ordered; //list of all vertices in the set for each trajectory
11    protected ArrayList<Integer> closeID; //store the ID of all vertices
12    protected ArrayList<Double> closeDist; //store the distance of all vertices
13    protected int totaltrj; //total trajectories in the set
14
15    /*
16     * Constructor
17     * @param id : id of the set
18     * @param totaltrj : total number of trajectories in the set
19     * @param FurthestEdge : the furthest edge
20     */
21    public MyFurSet(int id,int totaltrj,MyEdge FurthestEdge) {
22        this.id = id;
23        this.totaltrj = totaltrj;
24        this.FurthestEdge = FurthestEdge;
25        set = new HashSet<MyVertex>();
26        ordered = new ArrayList<ArrayList<Integer>>();
27        for (int i=0;i<totaltrj;i++) ordered.add(new ArrayList<Integer>());
28        closeID = new ArrayList<Integer>(totaltrj);
29        closeDist = new ArrayList<Double>(totaltrj);
30        for (int i = 0;i <totaltrj;i++) {
31            closeID.add(-1);
32            closeDist.add(Double.MAX_VALUE);
33        }
34    }
35
36 }
```

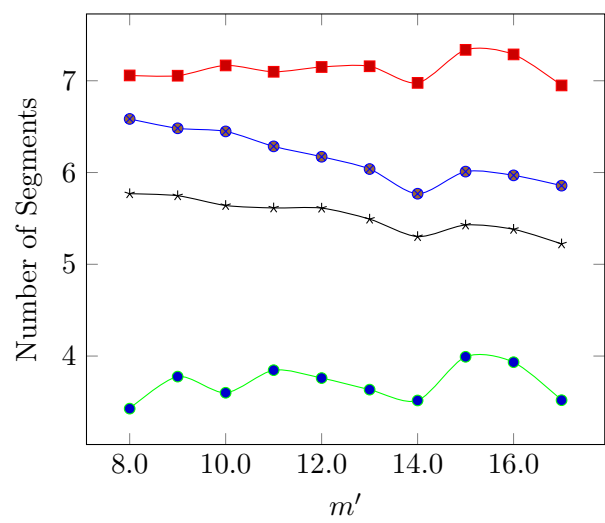

LAMPIRAN B

HASIL EKSPERIMEN

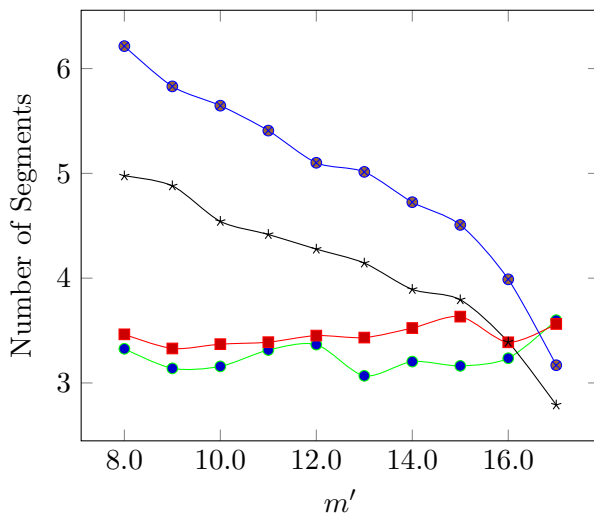
Hasil eksperimen berikut dibuat dengan menggunakan TIKZPICTURE (bukan hasil excel yg diubah ke file bitmap). Sangat berguna jika ingin menampilkan tabel (yang kuantitasnya sangat banyak) yang datanya dihasilkan dari program komputer.



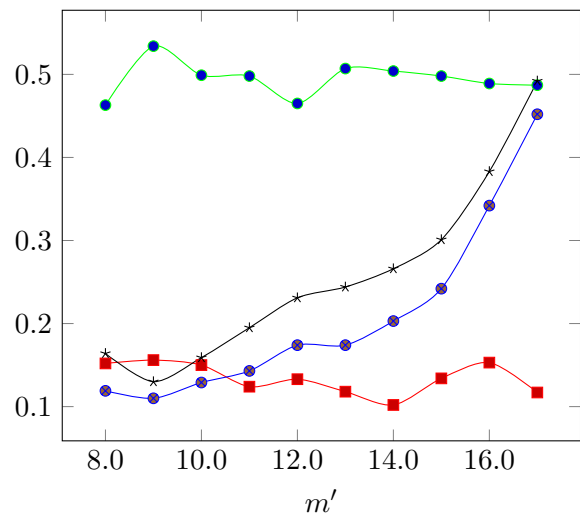
Gambar B.1: Hasil 1



Gambar B.2: Hasil 2



Gambar B.3: Hasil 3



Gambar B.4: Hasil 4